ETHEREUM PEERDAS · CODING SCHEME COMPARISON

Benchmarking 1D-RS, 2D-RS, and BC codes

How the code works, what it costs, and what it tolerates

Benchmarking 1D-RS, 2D-RS, and Block-Circulant codes

A controlled comparison of erasure-coding schemes as candidates for Ethereum's data-availability sampling: one-dimensional Reed-Solomon (what PeerDAS uses today), two-dimensional Reed-Solomon, and Block-Circulant (BC) codes at three parameterisations. All of them run over the same field, on the same blob, under the same erasure model, and are judged by the same success criterion.

The question each scheme has to answer is the same: **if a fraction of the encoded data goes missing, can the original blob still be reconstructed exactly — and what did that cost in storage and in decode time?**

This revision changes two things about the comparison itself, both aimed at making it fair rather than at making any scheme look better. 2D-RS now runs at 1.89× storage overhead instead of 4×, and BC now runs at parameterisations whose minimum distance is optimal rather than five. The practical effect is that every scheme in the table below stores roughly twice its blob, so differences in what they tolerate are differences between the codes, not between their budgets.

How the code works

The whole experiment is five moving parts. Each is a separate file, and the benchmark drives them without reimplementing any of their logic.

Packing a blob (`blob_generator.py`). An Ethereum blob is 4096 field elements of the BLS12-381 scalar field. Each element is 32 bytes on the wire but only 31 are usable, because the value must stay below the field modulus — so one blob carries 126,976 bytes of application data. This is the unit everything else operates on: a list of 4096 field elements, and a function to turn it back into the original bytes.

1D-RS (`1D_RS_Galois.py`). The key idea is that the blob's 4096 elements are not raw data to be copied — they are the *values of a single polynomial* sampled at 4096 fixed points. Encoding recovers that polynomial (an inverse NTT turns 4096 values into 4096 coefficients), pads it with zeros, and evaluates it again at 8192 points (a forward NTT). The result is 8192 symbols describing the same polynomial. Since a polynomial of degree below 4096 is pinned down by *any* 4096 of its values, any 4096 surviving symbols reconstruct everything. Decoding builds an erasure-locator polynomial whose roots sit exactly at the missing positions, multiplies through by it, inverts the transform, and divides the locator back out — recovering the original polynomial in one pass. Crucially it first checks whether the number of erasures exceeds `n−k`; if so it gives up immediately rather than attempting anything.

2D-RS (`2D_RS_Galois.py`). The same interpolate-and-extend step, applied twice. The blob is reshaped into a 64×64 grid; every row is extended, then every column of that result is extended. Because the original grid is a single bivariate polynomial, every row *and* every column of the result is independently a valid 1D-RS codeword. Decoding exploits this: it sweeps all columns, then all rows, repeatedly. A line holding too many erasures to fix alone often becomes fixable once the perpendicular sweep has repaired some of its cells, so repairs cascade across the two axes until nothing more can be done.

What changed here is the extension factor. A product code's rate is the *product* of its two component rates, so doubling both axes — the previous 64→128 setting — gives rate 1/4 and 4× storage. That made the earlier comparison unfair: 2D-RS was being judged at the same erasure rate as two codes spending half as much. Each axis now extends 64→88 instead, a component rate of 8/11 per axis and 0.529 overall, which is 1.89× storage. Exactly 0.5 turns out to be unreachable: rate 0.5 would need the codeword to be 8192 cells, a power of two forces both grid dimensions to be powers of two, and each axis must at least double to carry any parity at all — which lands on 16384, not 8192. 88 works because $88 = 8 \times 11$ and 11 divides $p-1$ for this field, so a length-88 NTT exists.

Block-Circulant (`BCcodeEthereum.py`). Structurally different: instead of one global polynomial, the message is cut into μ small blocks of ω symbols, and each block gets ρ parity symbols computed from its own information plus that of λ consecutive neighbours, wrapped around a ring. Overhead is 2×, the same as 1D-RS.

The parameterisation matters more than it first appears. Minimum distance is $d = \lambda\rho + 1$, so the previously benchmarked ω=ρ=λ=2 with μ=2048 has $d = 5$: it is *guaranteed* to correct only four erasures in an 8192-symbol codeword, and everything above that was luck about how the mask happened to fall. The three rows swept here instead set $\rho = \omega$ and $\lambda = \mu$, which makes every parity block depend on *all* information blocks and lifts d to $4097 = n - k + 1$. That is the Singleton bound: as **codes** these are MDS, exactly as good as Reed-Solomon.

What stays local is the **decoder**. `DecCBC[2]` resolves erasures within a single local block (at most ρ of them) or across an adjacent pair (at most 2ρ), and never looks at the codeword as a whole. Separating the code from the decoder is what this sweep measures.

The harness (`benchmark_1D_2D_BC.py`). Each scheme is wrapped in a small adapter exposing the same five operations — `encode`, `random_mask`, `corrupt`, `decode`, `extract_message` — so the sweep loop never needs to know that 2D-RS's codeword is a grid while BC's is a flat vector. The adapters call the real functions in the files above; no coding logic is duplicated, so fixing one of those files changes the benchmark's results automatically.

The experimental pipeline

Every measurement follows the same seven steps, and only two of them differ between schemes:

1. **Generate** 126,976 random application bytes.
2. **Pack** them into one blob of 4096 field elements, the real EIP-4844 way.
3. **Encode** — *scheme-specific* — expanding 4096 elements into a codeword of `n` elements.
4. **Erase** — withhold each codeword position independently with probability equal to the target rate, simulating samples that peers failed to provide.
5. **Decode** — *scheme-specific* — attempt to reconstruct the full codeword from what survived.
6. **Extract** the original 4096 elements back out of the recovered codeword.
7. **Verify** by unpacking to bytes and comparing against step 1.

Steps 1, 2, 4, 6 and 7 are byte-identical across every scheme. That is what makes the comparison fair: any difference in the results has to come from step 3 or step 5, which is precisely what is being compared.

Step 7 is the one worth dwelling on. Success is defined as *the recovered bytes equal the original bytes* — not as "the decoder reported success", and not even as "the field elements matched". That distinction is not pedantry: one of these decoders turned out to report success on data it had silently corrupted, and only the byte comparison caught it.

What is held identical

- **Field** — the real BLS12-381 scalar field.
- **Message** — the same 4096 elements, packed from the same bytes, fed to every encoder. Not one random blob per scheme.

- **Erasure model** — independent per-position withholding at a shared target rate. 1D-RS and all three BC rows share `n = 8192`, so on each trial they are corrupted at the *same literal positions*, not merely the same rate. 2D-RS has `n = 7744` and draws its own masks at the same rate.
- **Success criterion** — the full byte-level round trip described above.

Storage is now nearly matched too, which it was not previously: 2.00× for 1D-RS and every BC row, 1.89× for 2D-RS. The small residual difference is reported rather than controlled away.

Method

Erasure rates are swept from 2% to 55%. Trial counts vary by scheme and rate: a scheme decoding against a formal budget check behaves deterministically well away from its threshold, so one trial there is representative, while near a threshold the outcome genuinely varies from draw to draw and needs several. Trials are therefore concentrated around each scheme's own transition — which is not the same rate for all of them. Each success rate carries a 95% Wilson confidence interval.

Code parameters and storage cost

All of them carry the same 124 KiB blob payload; they differ in how much they store to protect it. `R = k/n` is the code rate, and `1−R` is the largest erasure fraction *any* code of that rate could survive.

	1D-RS	2D-RS	BC-1024	BC-512	BC-256
Message k	4096	4096	4096	4096	4096
Codeword n	8192	7744	8192	8192	8192
Code rate R	0.500	0.529	0.500	0.500	0.500
Storage overhead	2.00×	1.89×	2.00×	2.00×	2.00×
Stored per blob	256 KiB	242 KiB	256 KiB	256 KiB	256 KiB
Erasure limit 1−R	0.500	0.471	0.500	0.500	0.500

The three Block-Circulant rows differ only in how the same 4096 message symbols are cut into blocks. Every row sets $\rho = \omega$ and $\lambda = \mu$, which is what makes them MDS: $d = \lambda\rho + 1 = 4097 = n - k + 1$ in every case, matching the Singleton bound.

	BC-1024	BC-512	BC-256
Information per block ω	1024	512	256
Parity per block ρ	1024	512	256
Local blocks μ	4	8	16
Overlap λ	4	8	16
Local code length n_0	5120	4608	4352
Minimum distance d	4097	4097	4097
Local solve size	1024×1024	512×512	256×256
Predicted single-block limit ρ/n_0	0.200	0.111	0.059

Encode and decode cost, from the same run. Decode times are the mean over trials that actually recovered the blob; a dash means nothing recovered at that rate, so there is no decode time to report.

	1D-RS	2D-RS	BC-1024	BC-512	BC-256
Encode	0.46s	2.63s	8.95s	6.63s	5.42s
Decode @ 2%	5.24s	2.34s	0.38s	0.42s	0.59s
Decode @ 5%	11.24s	2.86s	0.84s	0.84s	0.94s
Decode @ 10%	22.02s	3.09s	1.73s	1.59s	—
Decode @ 20%	40.18s	5.26s	3.59s	—	—
Decode @ 35%	65.75s	6.03s	—	—	—
Decode @ 48%	83.94s	—	—	—	—

Results

Recovered trials at each erasure rate. Trial counts differ by scheme and by rate — more where a scheme is near its own threshold and the outcome genuinely varies, fewer where it does not — so each cell shows recovered/attempted rather than a bare percentage.

ERASURE RATE	1D-RS	2D-RS	BC-1024	BC-512	BC-256
2%	1/1	3/3	4/4	4/4	4/4
5%	1/1	3/3	4/4	8/8	8/8
8%	1/1	3/3	4/4	8/8	0/8
10%	1/1	3/3	4/4	8/8	0/8
12%	1/1	3/3	4/4	0/8	0/8
15%	1/1	3/3	8/8	0/8	0/8
20%	1/1	3/3	8/8	0/4	0/4
25%	1/1	3/3	0/8	0/4	0/4
30%	1/1	3/3	0/4	0/4	0/4
35%	1/1	5/5	0/4	0/4	0/4
38%	2/2	0/5	0/4	0/4	0/4
40%	2/2	0/5	0/4	0/4	0/4
45%	2/2	0/3	0/4	0/4	0/4
48%	3/3	0/3	0/4	0/4	0/4
50%	0/3	0/3	0/4	0/4	0/4
52%	0/3	0/3	0/4	0/4	0/4
55%	0/2	0/3	0/4	0/4	0/4

Across all 340 trials, no scheme returned corrupted data while reporting success: silent corruption was zero everywhere (0/27, 0/57, 0/80, 0/88, 0/88). Before the decoder fix described below, BC's figure was 31 of 40.

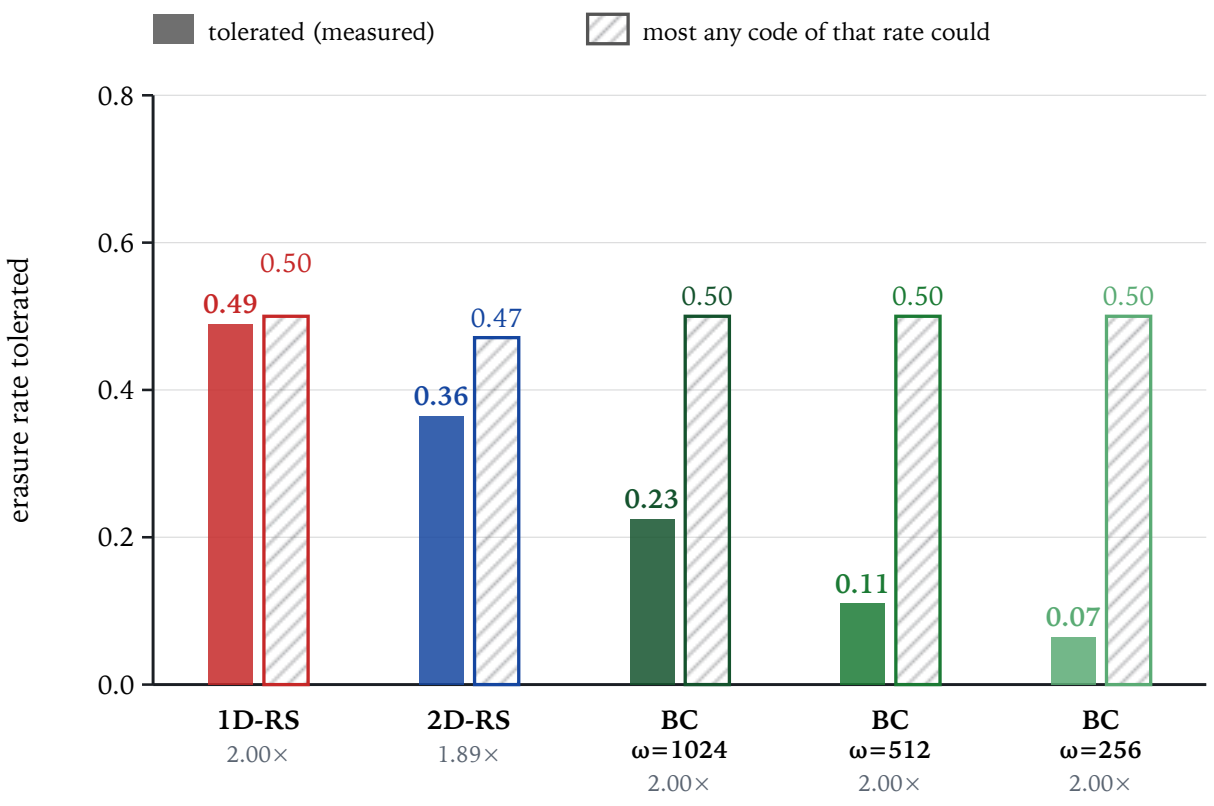


Fig. 1. What each scheme actually tolerated (solid) beside the most any code of the same rate could tolerate (hatched). Storage is now nearly matched, so these ceilings are nearly the same too and the bars can be read against each other directly. 1D-RS reaches its ceiling exactly — that is what being MDS means. 2D-RS gets about four fifths of the way to its own. The BC rows are MDS codes with the same 0.50 ceiling as 1D-RS, so the gap is entirely the decoder's. Solid bars are the midpoint of the measured transition from full recovery to none.



Fig. 2. The Block-Circulant parameter sweep. All three rows are MDS codes: $d = 4097 = n - k + 1$, so as codes they could each tolerate half the codeword being erased, the dashed line. What they actually tolerate is set by the decoder, which never looks beyond one local block. Because $\lambda = \mu$, a block's support holds the whole message plus its own parity, so a block survives while $4096x + \rho x \le \rho$, i.e. $x \le \rho/n_0$. The open markers are that prediction; the bars are what was measured. Larger local blocks tolerate more, and the local system they solve grows with them: $\rho \times \rho$, which generic elimination would solve in ρ^3 time. Exploiting the fact that it is a Vandermonde matrix brings that to ρ^2, which is what makes $\omega = 1024$ measurable.

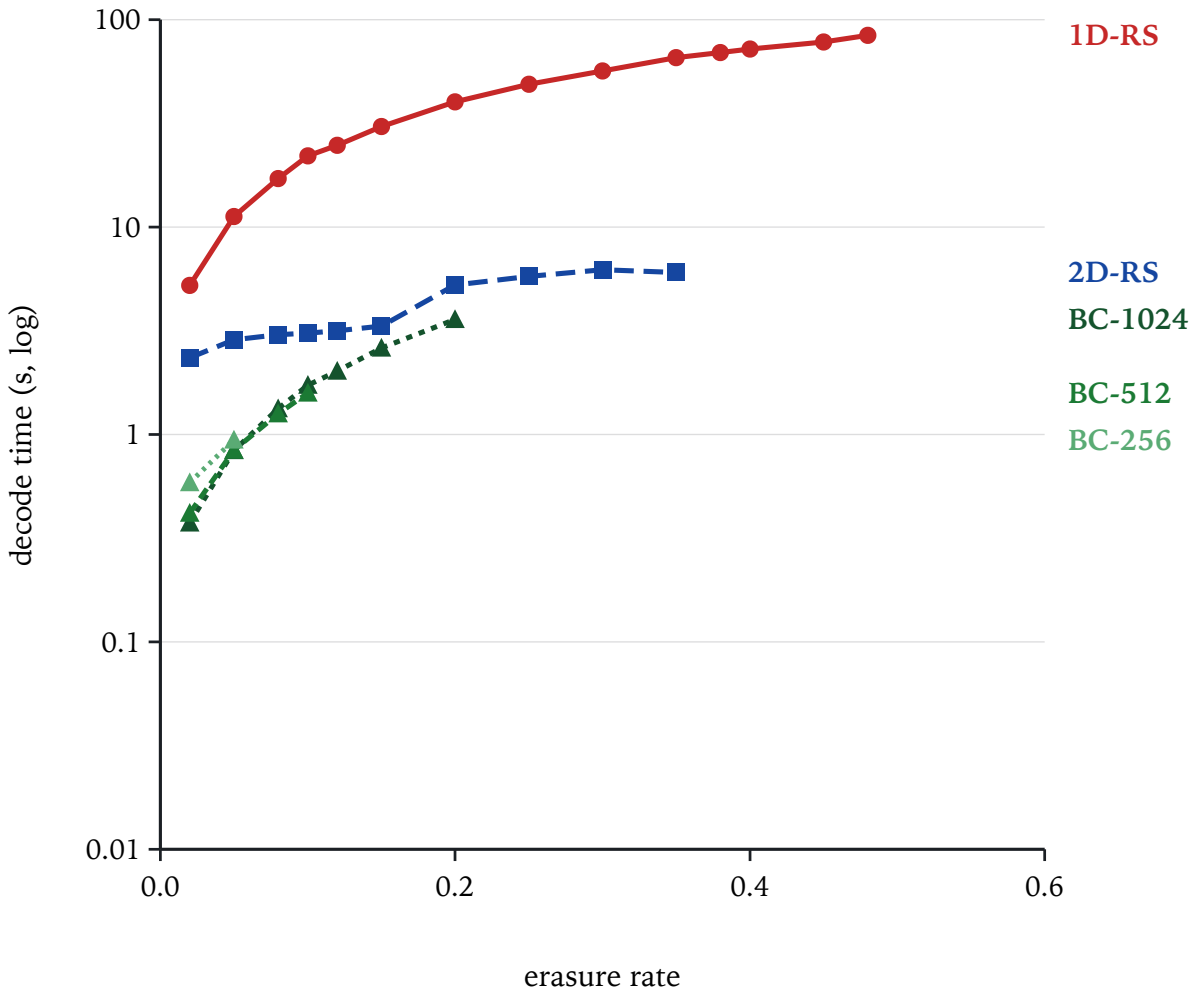


Fig. 3. Decode time against erasure rate, log scale. Only trials that actually decoded are plotted: above its threshold a scheme rejects the erasure pattern on a budget check and returns in under a millisecond having attempted nothing, which is not a decode time. 1D-RS pays the most because it divides one 8192-length polynomial; 2D-RS divides short ones instead. The three BC rows cost about the same as each other at any given rate, because a local system carries roughly $x \cdot n_0$ unknowns and n_0 is dominated by the same 4096 message symbols in all three; BC-1024 looks slowest only because it is still working at rates where the other two have already stopped. Every BC decode measured here is cheaper than 1D-RS at the same rate.

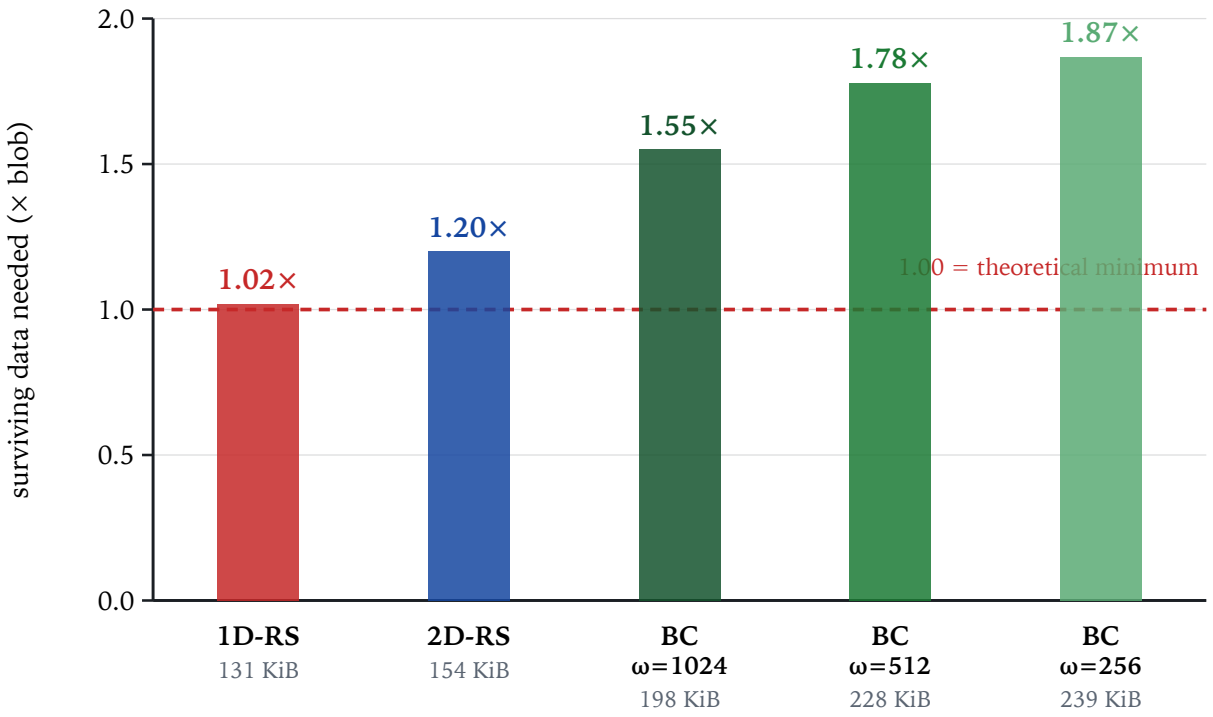


Fig. 4. How much surviving data each scheme needs to rebuild one blob, as a multiple of the blob itself. This is a different question from the tolerance figure, and it is the one that matters for data-availability sampling, where a node must actually fetch those symbols. 1D-RS needs exactly one blob's worth — the theoretical floor, since no code can rebuild 124 KiB from less than 124 KiB. Everything else needs more, and a scheme that tolerates a lower erasure rate necessarily needs more surviving data at the same storage cost.

What the results show

1D-RS behaves exactly as Reed-Solomon theory says it must. It recovers everything up to 48% and nothing from 50% upward. The invariant is exact rather than approximate: across every 1D-RS trial in the sweep, recovery succeeded if and only if the drawn erasure count was at most $n-k = 4096$, with no exceptions. The partial results at the threshold are not the decoder being unreliable — they are the random mask landing on either side of a hard line. Being MDS, 1D-RS extracts essentially all of its theoretical budget.

2D-RS, judged fairly this time, comes second. It holds up to 35% and fails from 38%, against a theoretical limit of 0.471 at its 1.89× storage — about 77% of its budget. The previous sweep had it tolerating a higher absolute rate than 1D-RS, but that was bought with 4× storage rather than with a better code. At comparable storage the cross-axis cascade does not

close the gap to MDS: lines that are individually over budget do get rescued once their partner axis repairs enough neighbours, but with only 24 spare cells per 88-cell line there is much less room for that cascade to start.

The Block-Circulant rows separate the code from the decoder, and the decoder is what limits them. All three are MDS — $d = 4097$, the same optimum 1D-RS achieves — so any 4096 surviving symbols determine the blob. What they actually reach is:

- **BC-1024** ($\omega=\rho=1024$, $\mu=\lambda=4$): recovers to 20%, fails from 25%; predicted $\rho/n_0 = 0.200$.
- **BC-512** ($\omega=\rho=512$, $\mu=\lambda=8$): recovers to 10%, fails from 12%; predicted $\rho/n_0 = 0.111$.
- **BC-256** ($\omega=\rho=256$, $\mu=\lambda=16$): recovers to 5%, fails from 8%; predicted $\rho/n_0 = 0.059$.

The measurements track the prediction closely, which is the useful part: the limit is not mysterious, it is arithmetic. Because $\lambda = \mu$, one local block's support contains the entire message plus that block's own parity, so a single block coming in under budget recovers everything at once. That happens while $4096x + \rho x \leq \rho$. Larger local blocks raise that limit, and the local system they solve grows with them: it is $\rho\times\rho$, which generic elimination handles in ρ^3 time. That is the trade-off the parameter table was built to expose, and it is also why the pairwise phase is unaffordable while the single-block phase is not — the latter's system is Vandermonde and drops to ρ^2.

Making these parameterisations runnable

The three BC rows could not be measured at all with the decoder as it stood. The reason is worth recording, because it is the same wall anyone re-running this will hit.

The original `gf_solve` is a Gauss-Jordan elimination written as a Python loop over rows. Its cost grows as the cube of the matrix dimension, and this field's arithmetic runs in `galois`' object mode at roughly 1.3 microseconds per element operation. At $\omega=\rho=2$ the local systems are 2×2 and none of that matters. At $\omega=\rho=1024$ they are 1024×1024, which is about a billion element operations — roughly 25 minutes for a single local solve, with μ of them per decoding round.

Three changes brought it into range, none of which alters what is computed:

The elimination step is now one broadcast update instead of a loop over rows. Same arithmetic, same element count, but the per-pivot dispatches into object-mode arithmetic drop from m to one. Measured: 1.34 → 0.55 microseconds per element operation, a 2.4× speedup.

The single-block solve now exploits the structure it always had. The local parity-check matrix is a GRS matrix, `H[r, c] = (c+1)^r`. Restricted to the erased columns and to its first e rows it is a square Vandermonde in distinct evaluation points — so it is always invertible, and it

can be solved in $O(e^2)$ by a node-polynomial argument rather than $O(e^3)$ by elimination. Measured at $e = 1024$: 0.87 seconds against an extrapolated 1441, about 1650×. This is what makes ω=1024 tractable.

The global parity-check matrix is no longer built. It is $(\mu\rho) \times n = 4096 \times 8192 = 33.5$ million field elements, and every row the decoder needs can be regenerated from (ρ, n_0) on demand.

A useful side effect: because the single-block system is provably square Vandermonde in distinct nodes, it cannot be underdetermined, so the class of bug described below cannot arise on that path at all.

The pairwise phase does not get a turn

`DecCBC[2]`'s second phase stacks two adjacent blocks' parity checks and solves them jointly. That system is *not* Vandermonde — the two row groups assign different evaluation points to the same shared columns — so it cannot use the fast solver and needs full elimination, at cubic cost.

Phase 2 only runs when phase 1 has stalled, and with $\lambda = \mu$ phase 1 stalls only once a support carries more than ρ erasures. Every row swept here has $\rho \geq 256$, so every pair that could possibly help carries more than 256 unknowns. The sweep applies a stated compute budget of 256 unknowns per pairwise system, identically to all three rows, and above that budget a pair is skipped, counted, and its positions left honestly flagged.

The consequence is that **all three rows are measured as single-phase decoders**, and this is a limit of affordable computation rather than of the code. The cost was measured directly on the 512-equation systems that ω=ρ=256 produces:

UNKNOWNS IN THE PAIR	ERASURE RATE IT CORRESPONDS TO	PER PAIR	PER DECODING ROUND (16 PAIRS)
276	6%	21.7s	5.8 min
369	8%	39.5s	10.5 min
461	10%	61.7s	16.4 min

That is for the *smallest* of the three rows, and one decoding round is not one decode — the loop repeats while it makes progress. At ω=ρ=1024 a full pair is 2048 equations in 2048 unknowns; at the 0.57 microseconds per field operation these timings imply, that is about 80 minutes for a single pair. Lifting the cap enough to change any result would take days per sweep.

The honest reading of the BC numbers is therefore: this is what the single-block phase of `DecCBC[2]` achieves. The pairwise phase would roughly double the budget from ρ to 2ρ, which the same arithmetic puts at 0.333 for ω=1024, 0.200 for ω=512 and 0.111 for ω=256 — still short of the 0.50 these codes permit.

The Block-Circulant decoder fix

The BC results are from a corrected decoder. The version originally benchmarked had two faults that together made its output untrustworthy rather than merely weak. Both were found at the earlier ω=ρ=λ=2 parameterisation, and the before/after figure below is measured there.

`gf_solve` fabricated values. The Gaussian elimination routine returned zero for any unknown whose column never received a pivot. To the caller that is indistinguishable from having genuinely solved for zero, so an underdetermined system silently produced a confident-looking wrong answer. It now raises `GFSolveError` unless the system has exactly one solution — every column pivoted, and no row reduced to `0 = nonzero`.

`dec_cbc_2`'s pairwise phase accepted systems it had not solved. Its guard is `0 < erasures ≤ 2ρ`, which is necessary but not sufficient. The equations it stacks are block *i*'s own ρ parity checks on top of block *i+1*'s — not the auxiliary `[2(ω+ρ), 2ω, 2ρ+1]` GRS code over the union of the two supports that the source paper's Algorithm 1 constructs. The stacked form is not MDS, so a split such as "three erasures inside block *i*'s support, one inside *i+1*'s" leaves two equations facing three unknowns. Combined with the `gf_solve` fault, the decoder wrote zeros into those positions *and cleared their erasure flags*, reporting success on corrupted data. It now skips any pair it cannot uniquely solve, leaving those positions honestly flagged.

The practical effect is stark. Measured before the fix, across 40 trials spanning 5–45% erasure, `dec_cbc_2` claimed success **every single time** and was wrong on 31 of them; it never once reported a failure. Its `remaining` flag — the signal a caller would naturally trust — carried no information about whether the data was correct. Had this benchmark trusted that flag instead of checking bytes, it would have reported BC at 100% success at every rate.

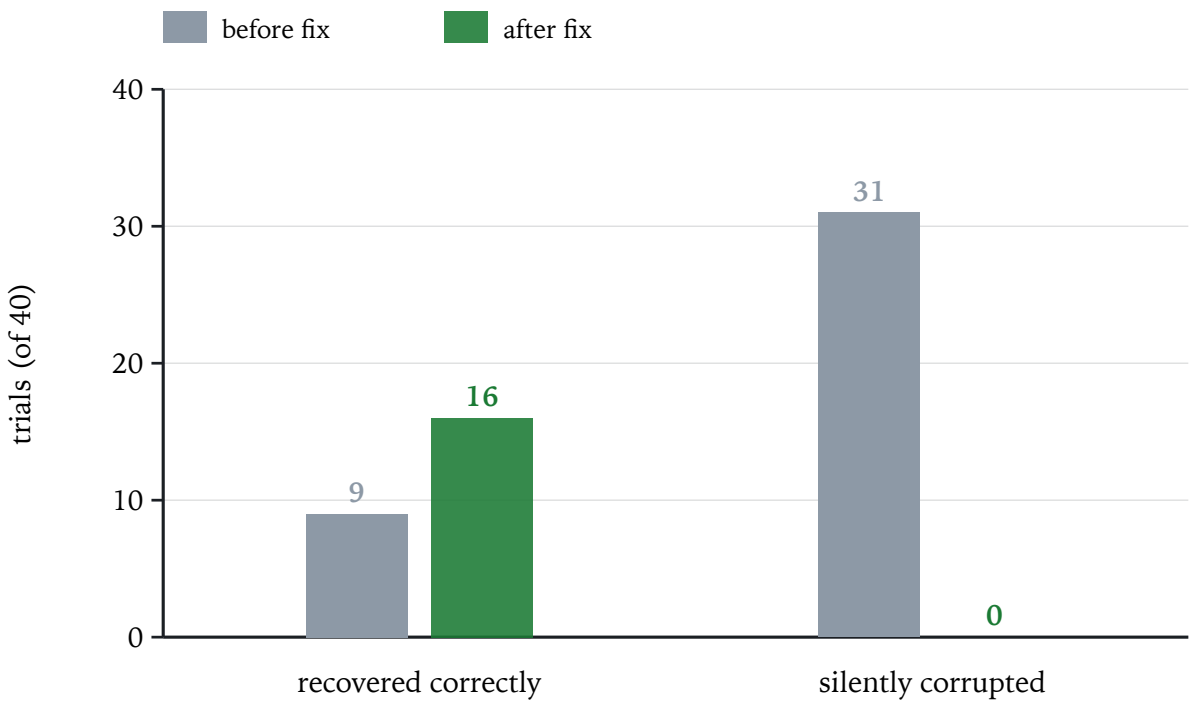


Fig. 5. The decoder fix, measured on the parameterisation it was found at (ω=ρ=λ=2, μ=2048) — *not* the rows swept above. The same 40 trials (5%, 15%, 25% and 45% erasure, ten each) run through the decoder before and after. Correct recoveries rose from 9 to 16, because the old decoder poisoned positions with fabricated zeros that blocked later rounds from resolving them. Silent corruptions — cases where the decoder reported success on wrong data — fell from 31 to zero. The remaining failures are honest: the decoder reports the positions it could not resolve.

A third change is a performance fix rather than a correctness one: the pairwise phase used to `break` and restart the single-block phase after every pair it repaired, paying a full scan of all local blocks per repair. It now continues through the remaining pairs in the same pass.

What none of this does is restore the correction radius the paper claims. Implementing the auxiliary GRS construction would let the pairwise phase handle *any* $\leq 2\rho$ erasures across two adjacent blocks, rather than only those solvable from the stacked parity checks — and, being a GRS code, it would also be solvable in $O(e^2)$ rather than $O(e^3)$, which would remove the compute wall described above. That is the single highest-value piece of remaining work on this file.

Reproducing

```
pip install galois numpy
python3 benchmark_1D_2D_BC.py
```

Per-trial results — erasure counts, decode times, success flags, and the decoder's own claim separately from ground truth — are written to `benchmark_results.json`.
